In [1]:
```
import pandas as pd
```

In [2]:
```
import numpy as np
```

In [3]:
```
import matplotlib.pyplot as plt
```

In [4]:
```
import seaborn as sns
```

In [5]:
```
from sklearn.model_selection import train_test_split
```

In [6]:
```
from sklearn.linear_model import LinearRegression
```

In [7]:
```
from sklearn.metrics import mean_squared_error, r2_score
```

In [1]:
```
import pandas as pd

df = pd.read_csv("AB_NYC_2019.csv")
df.head()
```

Out[1]:

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	l
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington	4
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	4
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem	4
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	4
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	4

In [2]:
```
import pandas as pd

df = pd.read_csv("AB_NYC_2019.csv")

print(df.shape)
df.head()
```

```
(48895, 16)
```

Out[2]:

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	l
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington	4
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	4
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem	4
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	4
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	4

In [3]:
```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48895 entries, 0 to 48894
Data columns (total 16 columns):
 #   Column                          Non-Null Count  Dtype  
---  ------                          --------------  -----  
 0   id                              48895 non-null  int64  
 1   name                            48879 non-null  object 
 2   host_id                         48895 non-null  int64  
 3   host_name                       48874 non-null  object 
 4   neighbourhood_group             48895 non-null  object 
 5   neighbourhood                   48895 non-null  object 
 6   latitude                        48895 non-null  float64
 7   longitude                       48895 non-null  float64
 8   room_type                       48895 non-null  object 
 9   price                           48895 non-null  int64  
 10  minimum_nights                  48895 non-null  int64  
 11  number_of_reviews               48895 non-null  int64  
 12  last_review                     38843 non-null  object 
 13  reviews_per_month               38843 non-null  float64
 14  calculated_host_listings_count  48895 non-null  int64  
 15  availability_365                48895 non-null  int64  
dtypes: float64(3), int64(7), object(6)
memory usage: 6.0+ MB
None
```

In [4]:
```
print(df.dtypes)
```

```
id                                 int64
name                              object
host_id                            int64
host_name                         object
neighbourhood_group               object
neighbourhood                     object
latitude                         float64
longitude                        float64
room_type                         object
price                              int64
minimum_nights                     int64
number_of_reviews                  int64
last_review                       object
reviews_per_month                float64
calculated_host_listings_count     int64
availability_365                   int64
dtype: object
```

In [5]:
```
print(df.describe())
```

```
                 id       host_id      latitude     longitude         price  \
count  4.889500e+04  4.889500e+04  48895.000000  48895.000000  48895.000000
mean   1.901714e+07  6.762001e+07     40.728949    -73.952170    152.720687
std    1.098311e+07  7.861097e+07      0.054530      0.046157    240.154170
min    2.539000e+03  2.438000e+03     40.499790    -74.244420      0.000000
25%    9.471945e+06  7.822033e+06     40.690100    -73.983070     69.000000
50%    1.967728e+07  3.079382e+07     40.723070    -73.955680    106.000000
75%    2.915218e+07  1.074344e+08     40.763115    -73.936275    175.000000
max    3.648724e+07  2.743213e+08     40.913060    -73.712990  10000.000000

       minimum_nights  number_of_reviews  reviews_per_month  \
count    48895.000000       48895.000000       38843.000000
mean         7.029962          23.274466           1.373221
std         20.510550          44.550582           1.680442
min          1.000000           0.000000           0.010000
25%          1.000000           1.000000           0.190000
50%          3.000000           5.000000           0.720000
75%          5.000000          24.000000           2.020000
max       1250.000000         629.000000          58.500000

       calculated_host_listings_count  availability_365
count                    48895.000000      48895.000000
mean                         7.143982        112.781327
std                         32.952519        131.622289
min                          1.000000          0.000000
25%                          1.000000          0.000000
50%                          1.000000         45.000000
75%                          2.000000        227.000000
max                        327.000000        365.000000
```

In [6]:
```
missing = df.isnull().sum()
```

In [7]:
```
missing_percent = (missing / len(df)) * 100
```

In [8]:
```
pd.DataFrame({
'Missing Values': missing,
'Percentage': missing_percent
}).sort_values(by='Percentage', ascending=False)
```

Out[8]:

	Missing Values	Percentage
last_review	10052	20.558339
reviews_per_month	10052	20.558339
host_name	21	0.042949
name	16	0.032723
neighbourhood_group	0	0.000000
neighbourhood	0	0.000000
id	0	0.000000
host_id	0	0.000000
longitude	0	0.000000
latitude	0	0.000000
room_type	0	0.000000
price	0	0.000000
number_of_reviews	0	0.000000
minimum_nights	0	0.000000
calculated_host_listings_count	0	0.000000
availability_365	0	0.000000

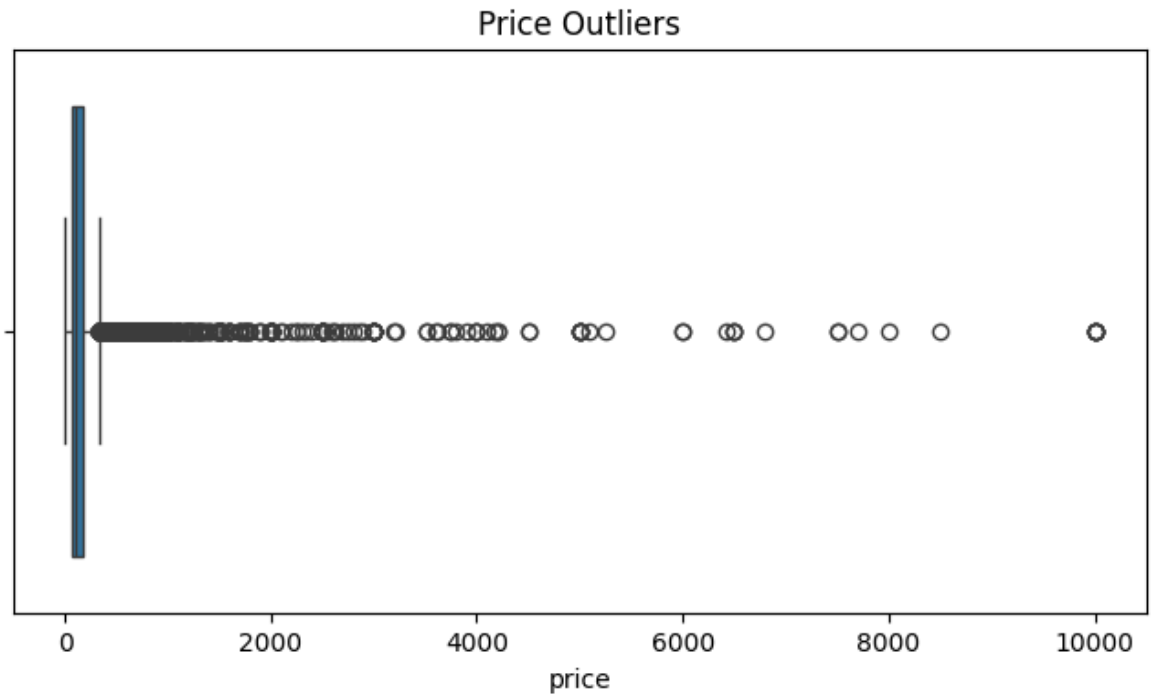
In [9]:
```
df.drop('last_review', axis=1, inplace=True)
```

In [10]:
```
df['reviews_per_month'] = df['reviews_per_month'].fillna(0)
```

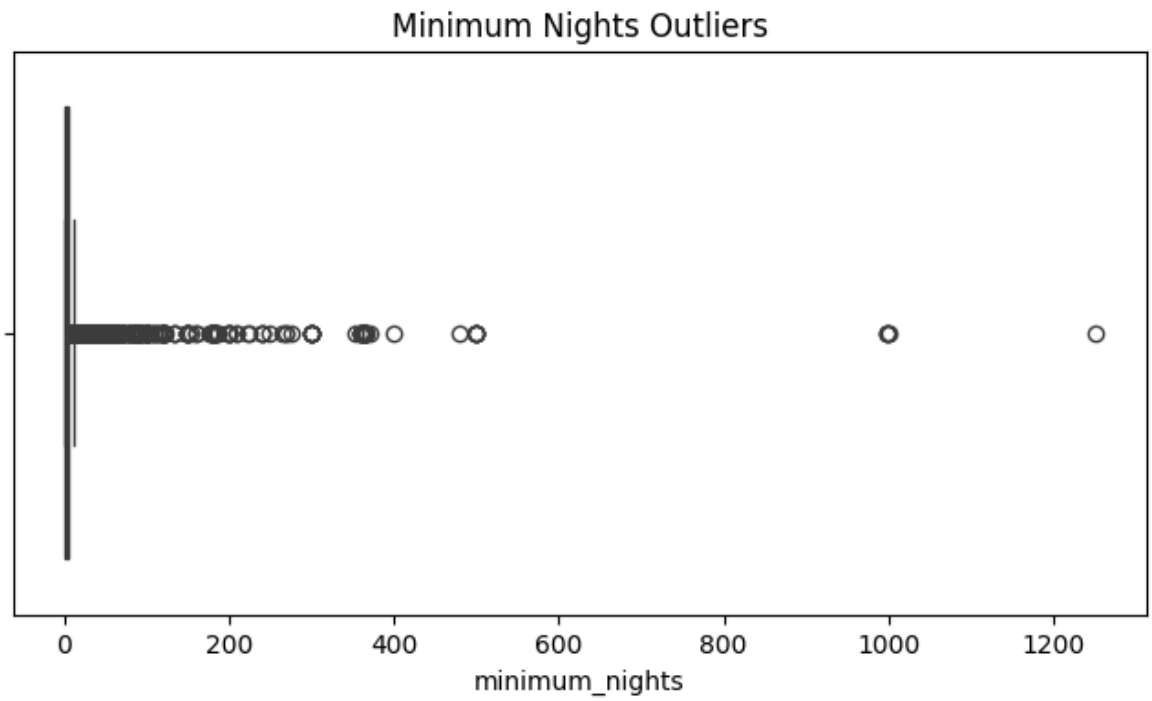
In [11]:
```
df.drop(['id', 'name', 'host_id', 'host_name'],
        axis=1,
        inplace=True)
```

In [12]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,4))
sns.boxplot(x=df['price'])
plt.title('Price Outliers')
plt.show()
```



In [13]:
```
plt.figure(figsize=(8,4))
sns.boxplot(x=df['minimum_nights'])
plt.title('Minimum Nights Outliers')
plt.show()
```



In [14]:
```
price_limit = df['price'].quantile(0.95)

df = df[df['price'] <= price_limit]
```

In [15]:
```
nights_limit = df['minimum_nights'].quantile(0.95)

df = df[df['minimum_nights'] <= nights_limit]
```

In [16]:
```
df.drop('neighbourhood',
        axis=1,
```

```
        inplace=True)
```

In [17]:
```
df = pd.get_dummies(
    df,
    columns=['neighbourhood_group', 'room_type'],
    drop_first=True
)
```

In [18]:
```
X = df.drop('price', axis=1)

y = df['price']
```

In [19]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

In [20]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)
```

Out[20]:
```
▾ LinearRegression
LinearRegression()
```

In [21]:
```
y_pred = model.predict(X_test)
```

In [22]:
```
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
import numpy as np

r2 = r2_score(y_test, y_pred)

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

print("R2 Score:", r2)
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
```

```
R2 Score: 0.47082773500020947
MAE: 38.54087491553849
MSE: 2834.959378892665
RMSE: 53.24433658984461
```

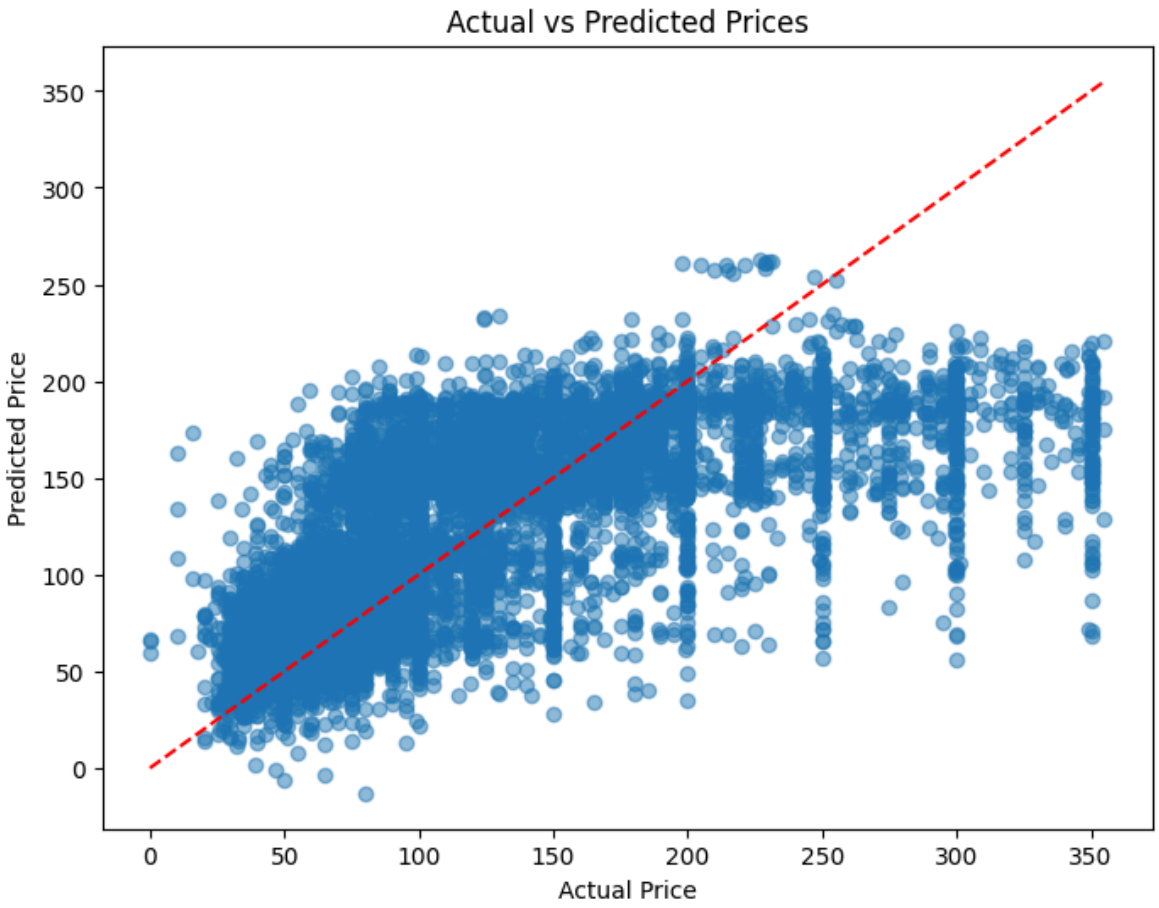
In [23]:
```
plt.figure(figsize=(8,6))
```

```
plt.scatter(y_test, y_pred, alpha=0.5)

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'r--'
)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Prices")

plt.show()
```



In [24]:

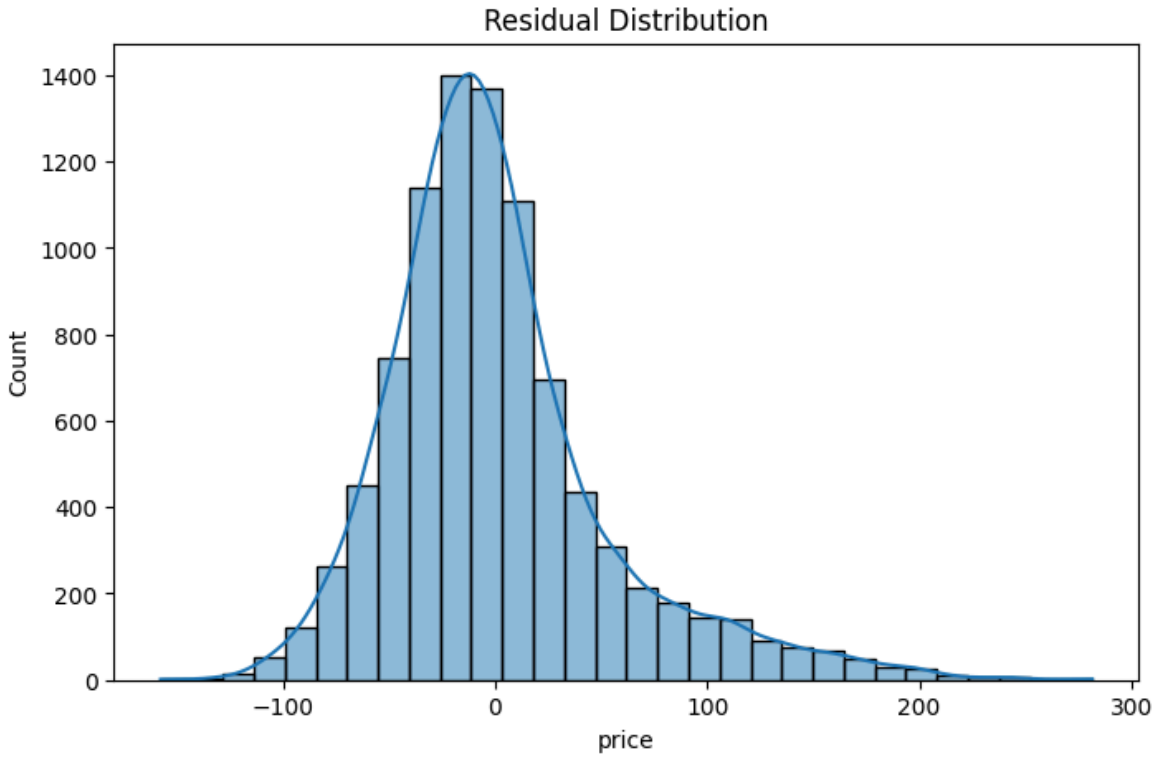
```
residuals = y_test - y_pred

plt.figure(figsize=(8,5))

sns.histplot(residuals,
             bins=30,
             kde=True)

plt.title("Residual Distribution")

plt.show()
```


Residual Distribution
Count
price
0
200
400
600
800
1000
1200
1400
−100
0
100
200
300